

Expiring Assets in Automated Market Makers

Kenan Wood
kewood@davidson.edu
Davidson College

Maurice Herlihy
mph@cs.brown.edu
Brown University

Hammurabi Mendes
hamendes@davidson.edu
Davidson College

Jonad Pulaj
jopulaj@davidson.edu
Davidson College

January 2024

Abstract

An automated market maker (AMM) is a state machine that manages pools of assets, allowing parties to buy and sell those assets according to a fixed mathematical formula. AMMs are typically implemented as smart contracts on blockchains, and its prices are kept in line with the overall market price by *arbitrage*: if the AMM undervalues an asset with respect to the market, an “arbitrageur” can make a risk-free profit by buying just enough of that asset to bring the AMM’s price back in line with the market.

AMMs, however, are not designed for assets that *expire*: that is, assets that cannot be produced or resold after a specified date. As assets approach expiration, arbitrage may not be able to reconcile supply and demand, and the liquidity providers that funded the AMM may have excessive exposure to risk due to rapid price variations.

This paper formally describes the design of a decentralized exchange (DEX) for assets that expire, combining aspects of AMMs and limit-order books. We ensure *liveness* and *market clearance*, providing mechanisms for liquidity providers to control their exposure to risk and adjust prices dynamically in response to situations where arbitrage may fail.

1 Introduction

An *automated market maker* (AMM) [7] is an automaton (usually implemented as a smart contract) that trades one electronic asset for another at rates set by a fixed mathematical formula. *Liquidity providers* (LPs) lend assets to the AMM in return

for a share of transaction fees and governance rights. Today, AMMs form a multi-billion dollar business [18]. The principal benefit of AMMs over more traditional order-book methods is that an AMM offers its assets at a take-it-or-leave-it price, so trades occur immediately, without need for complex bidding strategies.

AMMs typically trade long-lived assets such as cryptocurrencies or stablecoins. In this paper, we investigate a decentralized exchange (DEX) that can be designed to trade *expiring assets*, those that have a fixed date after which they cannot be resold. For example, a flight ticket from New York to Paris that departs on Monday cannot be sold on Tuesday. Expiration can have complicated effects on price: a last-minute airline ticket may sell at a premium, but a last-minute ticket for a Broadway show may sell at a discount.

Parties can play three distinct roles. *Producers* sell (expiring) *tokens* through the DEX. For ease of exposition, all tokens expire at the same well-known time and date. *Consumers* use (electronic) *cash* to purchase those tokens through the DEX. Cash has a stable value, and acts as the numéraire. (Some parties may combine both roles to act as *arbitrageurs*, buying and selling tokens with the effect of aligning the DEX token price with an exogenous market price.) Finally, *liquidity providers* (LPs) are market makers, buying tokens from producers and selling them to consumers in return for fees.

For most of the DEX’s lifetime, it acts like a standard automated market maker. Producers and consumers sell and buy tokens at a take-it-or-leave-it price determined by the formula specific to the DEX. All transactions are executed immediately. If there is a known, exogenous “market” where tokens can also be bought and sold, then arbitrageurs keep the DEX price in agreement with the market price. Liquidity providers bootstrap the DEX by loaning assets, and they profit by taking a fee for each transaction. When the system shuts down, its remaining cash is returned to the LPs.

As the expiration date approaches, standard market mechanisms may fail. For some unknown but brief time before token expiration, token supply may vanish: producers stop selling new tokens on the DEX, no matter the price. (For example, airlines cannot create more seats for an imminent Monday flight to Paris.) Similarly, token demand may vanish: consumers stop buying tokens on the DEX, no matter the price. (Hotel rooms in the Caribbean are not going to be booked by people that cannot get there on time) In addition, airline tickets, hotel rooms, and similar expiring assets have prices that can vary highly in response to exogenous events that may be harder to predict, such as weather, geopolitics, etc. Under these circumstances, the DEX can no longer rely on arbitrage to keep the market efficient.

In particular, conventional AMMs fail under those circumstances. Normally, if an AMM’s price for tokens (in cash) rises above the market price, then an arbitrageur would buy (or create) new tokens and sell them to the AMM, thus collecting a risk-free profit and bringing the AMM’s token price back to the market rate. If supply vanishes, however, each token purchase will drive the AMM price up, and eventually

the AMM’s token price will rise just above the market price, and the remaining tokens will expire unsold, even though both the consumers and the LPs would have been better off selling those tokens at a lower price. From this we learn that a DEX for expiring assets cannot rely exclusively on arbitrage to keep prices in line with the market.

In an alternative scenario, if demand vanishes, token producers will dump their expiring tokens on the AMM. Of course, such panicky selling will cause the AMM price to fall, but the token producers will realize at least some cash for their tokens. But these trades come at the expense of the LPs who loaned the cash that is being replaced with soon-to-be worthless tokens. From this we learn that AMMs for expiring assets must provide some way for LPs to intervene to limit their exposure to market distortions due to expiration.

This paper describes the design of a decentralized exchange (DEX) for tokens that expire, with the following design goals.

- *Market clearing*: at expiration time, it is never the case that there is a producer willing to sell a token at some price ¹, and a consumer willing to buy at that price, yet that token remains unsold,
- *Instant gratification*: each asset is bought and sold either immediately (during normal operations) or after a known delay (as expiration approaches), and
- *Incentives*: each participant has an incentive to participate.

This paper’s contribution is the design and analysis of a novel DEX for trading assets that expire. Our analysis points out ways in which conventional AMM design fails to address issues raised by expiring tokens, and we propose the following mechanisms to augment conventional AMMs to handle expiring assets.

- LPs can respond to unforeseen fluctuations in token supply and demand – in particular, in cases resulting in arbitrage failure – by directly intervening to change the shape of the AMM’s curve.
- LPs can limit its exposure to demand failure by setting a token price below which their cash cannot be used to buy tokens (similar to Uniswap V3’s concentrated liquidity).
- The AMM is augmented with a pair of order books to ensure market clearing, even in situations of supply and demand failure. For example, just before the tokens expire, any tokens remaining in the DEX are auctioned off, prioritizing LPs in order of cumulative participation.

LPs are assumed to be rational. They may disagree on market predictions, but they do not act maliciously against one another.

¹A producer may have a minimum price even for expiring tokens. For example, a hotel might not rent a room for less than the cost of cleaning that room.

2 Related Work

Today, the most popular automated market maker is *Uniswap* [1, 4, 13, 19], a family of constant-product AMMs. Originally trading between ERC-20 tokens and ether cryptocurrency, later versions added direct trading between pairs of ERC-20 tokens, and allowed liquidity providers to restrict the range of prices in which their assets participate. *Bancor* [14] AMMs permit more flexible pricing schemes, and later versions [6] include integration with external “price oracles” to keep prices in line with market conditions. *Balancer* [15] AMMs trade across more than two assets, based on a *constant mean* formula that generalizes constant product. *Curve* [11] uses a custom curve specialized for trading *stablecoins*, maintaining low slippage and divergence loss as long as the stablecoins trade at near-parity. The formal model for AMMs used here is adapted from Engel and Herlihy [12].

Xu *et al.* [18] and Bartoletti *et al.* [7] provide informative overviews on AMM protocols. Angeris and Chitra [2] introduce a *constant function market maker* model and focus on conditions that ensure that agents who interact with AMMs correctly report asset prices.

Aoyagi [5] analyzes strategies for constant-product AMM liquidity providers in the presence of “noise” trading, which is not intended to move prices, and “informed” trading, intended to move the AMM to the stable point for a new and more accurate valuation. Angeris *et al.* [3] propose an economic model relating how the curvature of the AMM’s function affects LP profitability in the presence of informed and noise traders. Bichuch and Feinstein [8] propose a general mathematical framework for AMMs, and Capponi *et al.* [9] analyze AMMs using a game theoretic model.

Our mechanism where liquidity providers can *freeze* their liquidity under adverse market conditions is similar to Uniswap V3’s [13] notion of *concentrated liquidity*, though the purpose and operational details are somewhat different.

Ramseyer *et al.* [17] describe several ways to integrate constant-function AMMs with batch auctions. Like the exchange described in this paper, their exchanges have hybrid structures, but their goals and techniques are substantially different, as they are concerned with broad economic properties, not with expiring assets.

3 Informal System Overview

Our DEX is composed of an AMM and two order books that operate under specific rules. We give now an operational overview of our DEX, starting with the AMM component. We point the reader to the appropriate sections containing detailed discussion as we present the concepts below.

An AMM owns and trades two kinds of assets: a stable asset X (informally called *cash*), and an expiring asset Y (informally called *tokens*). These assets are loaned to the AMM by *liquidity providers* (LPs). As explained later, LPs can be *active* or *inactive*.

The AMM tracks a pair $(x, y) \in \mathbb{R}_{>0}^2$, where x (respectively y) is the amount of X (respectively Y) that the AMM offers for trading.

Any party ℓ can become a liquidity provider by lending assets to the AMM. In return, ℓ receives a *share* $s_\ell \in (0, 1]$ proportional to its contribution. In departure from “pure” AMM designs, an LP can dynamically influence AMM prices to a degree proportional to its share. The particular protocol and curve design is discussed in Section 5.

LPs can withdraw liquidity at well-defined times, called *epoch boundaries*. LPs can make withdrawals denominated in both types of assets, or in only the stable asset X . LPs can *freeze* their liquidity, in a manner comparable to Uniswap V3’s notion of concentrated liquidity [13]. An LP ℓ can set a minimum spot price for Y tokens in terms of X cash. If the AMM’s spot price for tokens falls below that LP’s minimum, that LP becomes *inactive* and its liquidity is not used in trades. Of course, LPs do not accrue fees while they are inactive. An LP can use freezing to protect itself if it fears that demand has collapsed, causing producers to flood the AMM with Y tokens no one will buy. If the price later rises above an LP’s minimum, that LP becomes active again. Details are discussed in Section 6.

To ensure market clearing, there are complementary buyer and seller order books that are primarily used in the following situations: just before the tokens expire, when there are not enough tokens in the system to execute any more cash-for-tokens trades (demand is too high), or when all LPs are inactive, so no more AMM trades can be executed temporarily. Details can be found in Section 7. If a consumer registers a below-spot-market bid (on Y tokens) on the buyer order book and is still present near the expiration time, that bid is executed with a second price auction. There is also a seller order book, (almost) symmetric to the buyer order book, where sellers offer to sell Y tokens that remain after the AMM token pool is exhausted. The combination of the AMM and the two order books acts like an AMM while normal supply-and-demand laws hold, but acts more like an order book (or auction) near the expiration date when those laws may fail.

We also include a mechanism to incentivize LP participation, so that the AMM (instant-gratification) component of the DEX remains live for a longer period of time. Of course, LPs can withdraw or freeze liquidity. However, if an LP ℓ chooses to take on more risk by participating more, they will be given priority in clearing their own tokens, and with a higher price, in the described market clearing auction. Thus, even if two LPs ℓ_1, ℓ_2 have the same share and hold the same number of tokens, if ℓ_1 participates in many more (and larger) trades than ℓ_2 , then ℓ_1 will receive a greater share of the auction proceeds than ℓ_2 . We discuss these details in Section 7.

3.1 Classifying Consumers

In standard AMM models (*e.g.*, Milionis *et al.* [16]), so-called *noise* traders buy tokens with the intent of consuming them when they expire, while *arbitrageurs* (sometimes

called *informed* traders) trade between the DEX and a centralized market to make a profit. These distinctions still apply when assets expire; but consumers, producers, and even LPs have important additional characteristics.

- *Price*: How much are customers willing to pay for tokens as expiration approaches? If demand outlasts supply, then producers may be unable to bring new tokens to market, driving up the price as time runs out. If supply outlasts demand, producers may flood the DEX with unwanted tokens, yielding a final price of (practically) zero. The design of the DEX should be able to accommodate both extremes as well as outcomes in between.
- *Urgency*: What kinds of risks influence consumer behavior? A consumer who waits until the last minute may not be able to buy a token if demand outstrips supply (*e.g.*, all flights to Paris on Monday are sold out). Alternatively, a last-minute consumer may receive a favorable deal if supply outstrips demand (*e.g.*, a last-minute bargain on a theater ticket). Consumers worried about missing out are said to have *high urgency*, while those willing to gamble on last-minute bargains have *low urgency*.

To summarize, there are three kinds of consumers:

- *Bargain hunters*: low urgency, low price. For example, a consumer who is willing to buy a last-minute ticket to a Broadway show if those tickets are cheap enough.
- *Normal customers*: high urgency, low price. These are conventional AMM (noise or informed) traders who want to execute their trades immediately at a take-it-or-leave it price.
- *High flyers*: high urgency, high price. For example, a consumer who suddenly needs to fly to an urgent business meeting in Paris on Monday.

We do not consider consumers with low urgency but high price. A symmetric classification can be applied to producers, but we focus mostly on consumers.

The last-minute consumer order book allows bargain hunters to bid on tokens at prices below the AMM's asking price. A bargain hunter unwilling to meet the AMM's asking price can register a bid on the last-minute consumer order book. That bid will be executed immediately if the AMM price falls to the bid price. Just before expiration, any unsold tokens will be distributed among the order book bidders via a second-price auction (or any other incentive-compatible auction mechanism).

Normal consumers, who want to buy tokens at market price right away, and normal producers, who want to sell tokens at market price right away both use the AMM component of the DEX.

High flyers, who need tokens at (almost) any price, will first go to the AMM. If the AMM is sold out of tokens (because, for example, expiration is near and supply

is exhausted), then a high-flyer can place an order on the last-minute consumer order book. Last-minute producers who consider the AMM price too low can monitor this order book and fulfill any satisfactory orders.

The last-minute producer and consumer order books ensure *market clearing*: just before expiration, no token remains unsold if its producer was willing to sell at a price some unfulfilled consumer was willing to pay.

As discussed below, additional mechanisms are necessary to serve the interests of LPs, who are also exposed to risks stemming from failures in either supply or demand as expiration approaches.

4 Mathematical Preliminaries

We now prepare to describe the system in proper detail, first quickly discussing preliminary notation and concepts that are necessary for a formal system presentation in the sections that follow.

For vectors $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ and $y = (y_1, \dots, y_n) \in \mathbb{R}^n$, we write $x \leq y$ provided $x_i \leq y_i$ for all $i \in [n]$, where $[n] = \{1, \dots, n\}$; if also $x \neq y$, we write $x \lneq y$. When referencing topological properties in this paper, we assume that any subset of $\mathbb{R}^n$ (for any $n \geq 1$) is given the subspace topology, unless otherwise specified. We define $\mathbb{R}_{>0} = \{x \in \mathbb{R} : x > 0\}$. A function $A : \mathbb{R}^n \rightarrow \mathbb{R}$ is *twice-differentiable* if all of its second partial derivatives exist and are continuous. We also say that A is *strictly increasing in each coordinate* if for all $x, y \in \mathbb{R}^n$ such that $x \lneq y$, we have $A(x) < A(y)$.

A set $\Omega \subseteq \mathbb{R}^n$ is *convex* if for all distinct $x, y \in \Omega$ and $t \in (0, 1)$, we have $tx + (1 - t)y \in \Omega$; if every such vector $tx + (1 - t)y$ is in the interior of Ω , we say Ω is *strictly convex*. It follows that any convex open set is strictly convex. Given a set $S \subseteq \mathbb{R}^n$, the *convex hull* of S , denoted $\text{conv } S$, is defined as the intersection of all nonempty convex sets $\Omega \subseteq \mathbb{R}^n$ that contain S . Suppose $\Omega \subseteq \mathbb{R}^n$ is convex. A function $f : \Omega \rightarrow \mathbb{R}$ is said to be *strictly convex* provided that for every distinct $x, y \in \Omega$ and $t \in (0, 1)$, it follows that $f(tx + (1 - t)y) < tf(x) + (1 - t)f(y)$.

5 Price Adjustment Mechanisms

As discussed in Section 3, our DEX has an AMM meant to provide an instant buying price for consumers. In a conventional constant-function AMM, token prices are kept consistent with the market by external *arbitrageurs*; if the token price rises above the market price, an arbitrageur may sell overpriced tokens to the AMM, making a risk-free profit. Conversely, if the AMM price falls below the market price, an arbitrageur may buy the bargain tokens from the AMM and resell them on the market. Arbitrage works well for a conventional AMM because there are sufficiently many tokens available to be traded, so that arbitrageurs are always able to trade

in the direction needed to bring the AMM price back in line with the market. But close to expiration date, token supply or demand may vanish, and the normal price regulation cannot take place. For example, airlines will not schedule new seats to Paris as the flight time becomes too close. Similarly it makes little sense to book a room in the Caribbean starting in one hour if the room is five hours away. In addition, airline tickets, hotel rooms, event seats, and similar expiring assets have prices that may be influenced by hard-to-predict, exogenous events, such as weather, geopolitical events, etc.

In conventional AMMs, LPs can deposit or withdraw liquidity, but are otherwise passive, allowing producers and consumers to set asset prices via trading. As noted, however, arbitrageurs may become unable to intervene to keep prices close to market levels as expiration approaches. So we propose a mechanism to allow LPs to intervene actively to adjust the curve when conventional methods are in danger of failing.

More specifically, in the period while expiration is far off, when the DEX acts like a conventional AMM, arbitrageur profits come at the expense of LPs, a cost known as *divergence loss*. The LPs effectively gamble that asset prices will be stable enough that the fees collected will outweigh any divergence loss. As expiration approaches, however, prices may become volatile and arbitrage may become ineffective, so LPs require the ability protect their investments by intervening directly to adjust prices (also to actively protect their capital – see Section 6), effectively assuming the role of conventional arbitrageurs, but without divergence loss.

The duration between when the DEX is started and when the token expires is divided into disjoint intervals called *epochs*, separated by *epoch boundaries*. There are two special epoch boundaries: t_e , when the tokens expire, and $t_r < t_e$, when the AMM halts and the last-minute auctions occur. Let T be the set of all epoch boundaries strictly before t_r .

We say that a *state curve* is a strictly decreasing homeomorphism $f : \mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$. If $x > 0$, we say that the pair $(x, f(x))$ is a *state* of f , and we denote the set of all states of f by $\text{state}(f)$. Operationally, we say that the AMM is in state $(x, y) \in \text{state}(f)$ if it contains x units of X and y units of Y . Trades change the amounts x and y of X and Y , respectively, so that (x, y) is always in $\text{state}(f)$.

We consider only a fixed subfamily of possible state curves. Let $0 < a < b$ be fixed positive lower and upper bounds. (We may wish to require that $a < 1$ and $b = \frac{1}{a}$ for symmetry, though this is not required.) For every $c \in [a, b]$ and possible state $(x_0, y_0) \in \mathbb{R}_{>0}^2$, define the state curve

$$f_{c,x_0,y_0}(x) = y_0 \left(\frac{x}{x_0} \right)^{-c}$$

for all $x > 0$. Then $(x_0, y_0) \in \text{state}(f_{c,x_0,y_0})$. Let L be the current set of *active* liquidity providers (whose assets are allocated to trading), and L' is the set of *inactive* liquidity providers (whose assets are frozen). The sets L and L' and their associated methods are formally defined in Section 6.

At each time in T , each liquidity provider $\ell \in L \cup L'$ chooses some $c_\ell \in [a, b]$ that informally represents the price ℓ believes is most economically efficient, with knowledge of the current state of the AMM. Each $c_\ell \in [a, b]$ is aggregated into a single $c \in [a, b]$ with some deterministic *aggregation algorithm* $\text{aggregate}()$ by computing $c \leftarrow \text{aggregate}()$ at each epoch boundary. The $\text{aggregate}()$ algorithm has parameters $(c_\ell)_{\ell \in L}$ and $(s_\ell)_{\ell \in L}$ ² and returns some element of $[a, b]$. For fixed L , since $\text{aggregate}()$ is deterministic, it can be viewed as a map

$$\text{aggregate} : [a, b]^L \times \left\{ (s_\ell)_{\ell \in L} \in [0, 1]^L : \sum_{\ell \in L} s_\ell = 1 \right\} \rightarrow [a, b],$$

where A^B is the set of all tuples indexed by B with values in A for any sets A, B . Now we define the soundness properties for an aggregation algorithm.

Definition 5.1. *Let $\text{aggregate}()$ be an aggregation algorithm; let L be the particular set of active LPs at some time, and consider the corresponding aggregation function. We say that $\text{aggregate}()$ is valid if the following axioms hold.*

1. *For each $\ell \in L$, the function aggregate has continuous partial derivatives with respect to c_ℓ . Additionally,*

$$\frac{\partial(\text{aggregate})}{\partial \log(c_\ell)} = s_\ell \cdot \text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}).$$

2. *If there exists some x such that $c_\ell = x$ for all $\ell \in L$, then*

$$\text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = x.$$

For brevity, let $c = \text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L})$.

The first condition implies that for a small fixed change in $\log c_\ell$ from an LP ℓ with share s_ℓ , the corresponding change in c is proportional to s_ℓ . We use a logarithmic scale since derivatives are limits of *additive changes*, and $\log$ is an isomorphism from the group $\mathbb{R}_{>0}$ under multiplication to the group $\mathbb{R}$ under addition. (Note that $c_\ell \in \mathbb{R}_{>0}$, so it only makes sense to give them a multiplicative structure.)

The second condition is a simple boundary condition to guarantee that if all LPs are in consensus with a particular value c , the aggregated constant is the common value c .

In our model, we define $\text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L})$ to return

$$\prod_{\ell \in L} c_\ell^{s_\ell}, \tag{1}$$

²We exclude the c_ℓ 's and shares of inactive liquidity providers because their share of liquidity is not well-defined, as it is not necessarily the same over both of the assets. See Section 6 for details.

the geometric mean of each c_ℓ weighted by the share $s_\ell \in [0, 1]$ each liquidity provider holds in the system. Interestingly, we show in the Appendix that this aggregation algorithm is the *unique* valid aggregation algorithm.

The state curve is implied from the global variables c, x_0, y_0 stored in the AMM. Until the next time in T in which liquidity providers may update their value of c_ℓ , we fix c as defined above. Note that when the state (x, y) changes from adding/removing liquidity, freezing/unfreezing liquidity, and liquidating (specified in Section 6), we update $x_0 \leftarrow x$ and $y_0 \leftarrow y$, and we must also update the state curve in these cases. This is necessary because the liquidity state (x, y) after completing any of these methods is no longer in the state space of the current state curve f_{c, x_0, y_0} . We omit updates to (x_0, y_0) in any pseudocode of these methods for simplicity of presentation.

5.1 State Curve Properties

Now, let us show that this state curve construction acts like an AMM. We use axioms proposed by Engel and Herlihy [12] that define an *AMM* as follows. We note that this definition is only used in this section, and other references to the AMM refer to the system model described in this paper.

Definition 5.2. *An AMM is a function $A : \mathbb{R}_{>0}^2 \rightarrow \mathbb{R}$ that satisfies the following:*

- *A is twice-differentiable;*
- *A is strictly increasing in each coordinate;*
- *For each $b \geq 0$, the set $\{(x, y) \in \mathbb{R}_{>0}^2 : A(x, y) \geq b\}$ is closed and strictly convex.*

Under this definition, the state space of A is the set $\{(x, y) \in \mathbb{R}_{>0}^2 : A(x, y) = 0\}$ and is denoted $\text{state}(A)$. This is the same notation as the state space of a state curve, but the definition of state space should be clear, according to whether the function is a state curve or an AMM. Henceforth, we use A to denote an AMM and f to denote a state curve. It is important to note the difference between an AMM and a state curve as defined previously; an AMM is a function $\mathbb{R}_{>0}^2 \rightarrow \mathbb{R}$, but a state curve is a function $\mathbb{R}_{>0} \rightarrow \mathbb{R}_{>0}$. An AMM captures more information than the set of states alone on a state curve alone. In particular, if A is an AMM, then we may construct its state curve by mapping $x > 0$ to the unique $y > 0$ such that $A(x, y) = 0$. However, the converse is not necessarily true as not every state curve can be extended to an AMM satisfying the above three properties. Observe that there is no operational difference between an AMM and a state curve since the set of states completely determines the behavior of the system. This motivates the following definition.

Definition 5.3. *Given state curve f , we say that f induces an AMM if there exists an AMM A such that $\text{state}(f) = \text{state}(A)$.*

We will prove that for any $(x_0, y_0) \in \mathbb{R}_{>0}^2$ and $c > 0$ that f_{c,x_0,y_0} induces an AMM. For the remainder of this section, fix $(x_0, y_0) \in \mathbb{R}_{>0}^2$ and $c > 0$. To prove that f_{c,x_0,y_0} induces an AMM, we let $A(x, y) = x^c y - x_0^c y_0$. We shall first prove that A is an AMM. It is easy to see the following two lemmas, and the third one is simple.

Lemma 5.1. *A is twice differentiable.*

Proof. Observe that $\frac{\partial A}{\partial x} = cx^{c-1}y$ and $\frac{\partial A}{\partial y} = x^c$. Then $\frac{\partial^2 A}{\partial x^2} = c(c-1)x^{c-2}y$ and $\frac{\partial^2 A}{\partial y^2} = 0$, which are both continuous. Since both second partial derivatives of A exist and are continuous, A is twice differentiable. $\square$

Lemma 5.2. *A is strictly increasing in each coordinate.*

Proof. If $x' > x > 0$ and $y > 0$, then since $c > 0$, $A(x', y) = (x')^c y - x_0^c y_0 > x^c y - x_0^c y_0 = A(x, y)$. Similarly, if $x > 0$ and $y' > y > 0$, then $A(x, y') = x^c y' - x_0^c y_0 > x^c y - x_0^c y_0 = A(x, y)$, as desired. $\square$

Lemma 5.3. *For any $b \geq 0$, the set $\{(x, y) \in \mathbb{R}_{>0}^2 : A(x, y) \geq b\}$ is closed and strictly convex.*

Proof. Let $b \geq 0$ and let $S = \{(x, y) \in \mathbb{R}_{>0}^2 : A(x, y) \geq b\}$. Since A is continuous and $[b, \infty)$ is closed, $S = A^{-1}([b, \infty))$ is closed because the preimage of a closed set under a continuous function is closed. To prove that S is strictly convex, it suffices to show that for any distinct $(x_1, y_1), (x_2, y_2) \in S$ and $t \in (0, 1)$, it follows that $A(t \cdot (x_1, y_1) + (1-t) \cdot (x_2, y_2)) > b$. To this end, notice that since $(x_1, y_1), (x_2, y_2) \in S$, we know $x_1^c y_1 \geq b + x_0^c y_0$ and $x_2^c y_2 \geq b + x_0^c y_0$. Thus, by the Weighted AM-GM Inequality [10, p. 74],

$$\begin{aligned} A(t \cdot (x_1, y_1) + (1-t) \cdot (x_2, y_2)) &= (tx_1 + (1-t)x_2)^c (ty_1 + (1-t)y_2) - x_0^c y_0 \\ &> (x_1^t x_2^{1-t})^c (y_1^t y_2^{1-t}) - x_0^c y_0 \\ &= (x_1^c y_1)^t (x_2^c y_2)^{1-t} - x_0^c y_0 \\ &\geq (b + x_0^c y_0)^t (b + x_0^c y_0)^{1-t} - x_0^c y_0 \\ &= b. \end{aligned}$$

This shows that S is a strictly convex set, as desired. $\square$

Finally, we must verify that f_{c,x_0,y_0} and A have the same state spaces.

Lemma 5.4. $\text{state}(f_{c,x_0,y_0}) = \text{state}(A)$.

Proof. Observe that

$$(x, y) \in \text{state}(f_{c,x_0,y_0}) \Leftrightarrow y = y_0(x/x_0)^{-c} \Leftrightarrow x^c y = x_0^c y_0 \Leftrightarrow (x, y) \in \text{state}(A).$$

Thus $\text{state}(f_{c,x_0,y_0}) = \text{state}(A)$. $\square$

As a consequence, we have the following.

Theorem 5.5. *The state curve f_{c,x_0,y_0} induces an AMM.*

Proof. In letting $A(x, y) = x^c y - x_0^c y_0$ as above, Lemmas 5.1, 5.2, and 5.3 imply that A is an AMM. By Lemma 5.4, it follows that f_{c,x_0,y_0} induces an AMM. $\square$

Since every state curve used in our price control model induces an AMM, it follows that long before the expiration time, within epochs, our construction satisfies all of the properties and corollaries in [12]. However, approaching the expiration t_e of the expiring token, the assumptions in [12] begin to break down as market conditions shift. Thus an additional mechanism – in particular, the market clearing mechanism in section 8 – is needed to clear the market in the case of extreme conditions, such as when all liquidity providers are inactive or no more tokens can be sold using the AMM component of the DEX.

Now we study the behavior of price and slippage with respect to the variable c and changes in liquidity states. In particular, we have the following definitions.

Definition 5.4 (Price and Slippage). *Given a state curve f_{c,x_0,y_0} and a particular state $(x, y) \in \text{state}(f_{c,x_0,y_0})$, the instantaneous price, or the spot price, at (x, y) is defined by*

$$p_{c,x_0,y_0}(x) = \frac{1}{-f'_{c,x_0,y_0}(x)}.$$

Additionally, we define the instantaneous slippage by

$$s_{c,x_0,y_0}(x) = p'_{c,x_0,y_0}(x) = \frac{f''_{c,x_0,y_0}(x)}{(f'_{c,x_0,y_0}(x))^2}.$$

Proposition 5.6. *Given a state (x_0, y_0) , the instantaneous price $p_{c,x_0,y_0}(x_0)$ and slippage $s_{c,x_0,y_0}(x_0)$ are strictly decreasing in c , for $c > 0$. That is, if $0 < c_1 < c_2$, then*

$$p_{c_1,x_0,y_0}(x_0) > p_{c_2,x_0,y_0}(x_0) \quad \text{and} \quad s_{c_1,x_0,y_0}(x_0) > s_{c_2,x_0,y_0}(x_0).$$

Proof. Observe that for all $c > 0$,

$$f'_{c,x_0,y_0}(x) = -cy_0 \left(\frac{x}{x_0}\right)^{-c-1} \cdot \frac{1}{x_0}, \quad f''_{c,x_0,y_0}(x) = c(c+1)y_0 \left(\frac{x}{x_0}\right)^{-c-2} \cdot \frac{1}{x_0^2},$$

so that

$$f'_{c,x_0,y_0}(x_0) = -c\frac{y_0}{x_0}, \quad f''_{c,x_0,y_0}(x_0) = c(c+1)\frac{y_0}{x_0^2}$$

Thus

$$p_{c_1,x_0,y_0}(x_0) = \frac{1}{-f'_{c_1,x_0,y_0}(x_0)} = \frac{x_0}{y_0 c_1} > \frac{x_0}{y_0 c_2} = \frac{1}{-f'_{c_2,x_0,y_0}(x_0)} = p_{c_2,x_0,y_0}(x_0).$$

Furthermore, for all $c > 0$,

$$s_{c,x_0,y_0}(x_0) = \frac{f''_{c,x_0,y_0}(x_0)}{(f'_{c,x_0,y_0}(x_0))^2} = \frac{c(c+1)\frac{y_0}{x_0^2}}{\left(-\frac{cy_0}{x_0}\right)^2} = \frac{c+1}{c} \cdot \frac{1}{y_0} = \frac{1}{y_0} \cdot \left(1 + \frac{1}{c}\right).$$

Therefore

$$s_{c_1,x_0,y_0}(x_0) = \frac{1}{y_0} \cdot \left(1 + \frac{1}{c_1}\right) > \frac{1}{y_0} \cdot \left(1 + \frac{1}{c_2}\right) = s_{c_2,x_0,y_0}(x_0).$$

□

Finally, we prove that adding and removing liquidity does not change the instantaneous price. As discussed in the next section, we require that the ratio x/y of active liquidity in the AMM stays constant during these operations. It is also insightful to prove that adding liquidity decreases instantaneous slippage, and removing liquidity increases slippage. Since adding and removing liquidity holds x/y constant, the following proposition shows these results.

Proposition 5.7. *Let $c > 0$, and consider a state $(x_1, y_1) \in \text{state}(f_{c,x_0,y_0})$ and some $(x_2, y_2) \in \mathbb{R}_{>0}^2$ such that $\frac{x_1}{y_1} = \frac{x_2}{y_2}$, where we fix $(x_0, y_0) \in \mathbb{R}_{>0}^2$. Then $p_{c,x_0,y_0}(x_1) = p_{c,x_2,y_2}(x_2)$. Furthermore, $s_{c,x_0,y_0}(x_1) > s_{c,x_2,y_2}(x_2)$ if and only if $y_1 < y_2$.*

Proof. Observe that

$$f'_{c,x_0,y_0}(x) = -cy_0 \left(\frac{x}{x_0}\right)^{-c-1} \frac{1}{x_0} = -c \cdot \frac{f_{c,x_0,y_0}(x)}{x}$$

for all $x > 0$. Because $(x_1, y_1) \in \text{state}(f_{c,x_0,y_0})$ and $(x_2, y_2) \in \text{state}(f_{c,x_2,y_2})$, we have

$$p_{c,x_0,y_0}(x_1) = \frac{1}{c \cdot \frac{f_{c,x_0,y_0}(x_1)}{x_1}} = \frac{x_1}{cy_1} = \frac{x_2}{cy_2} = \frac{1}{c \cdot \frac{f_{c,x_2,y_2}(x_2)}{x_2}} = p_{c,x_2,y_2}(x_2).$$

We also have

$$f''_{c,x_0,y_0}(x) = c(c+1)y_0 \left(\frac{x}{x_0}\right)^{-c-2} \cdot \frac{1}{x_0^2} = c(c+1) \cdot \frac{f_{c,x_0,y_0}(x)}{x^2},$$

so that $s_{c,x_0,y_0}(x) = \frac{c+1}{c} \cdot \frac{1}{f_{c,x_0,y_0}(x)}$ for all $x > 0$. It follows that

$$s_{c,x_0,y_0}(x_1) = \frac{c+1}{c} \cdot \frac{1}{y_1}, \quad s_{c,x_2,y_2}(x_2) = \frac{c+1}{c} \cdot \frac{1}{y_2}$$

since $(x_1, y_1) \in \text{state}(f_{c,x_0,y_0})$ and $(x_2, y_2) \in \text{state}(f_{c,x_2,y_2})$. Hence $s_{c,x_0,y_0}(x_1) > s_{c,x_2,y_2}(x_2)$ if and only if $y_1 < y_2$. □

6 Freezing Liquidity and Related Methods

With expiring assets (and exogenous events that affect price), it is important to avoid that LPs have their cash (X) drained upon a sudden decrease in demand for tokens (Y). In this section, we discuss *liquidity freezing*, a tool allowing an LP to alter how much of its liquidity the AMM can use for trading under current market conditions.

LPs add liquidity in a way almost identical to AMMs such as Uniswap [1]. If (x, y) is the current liquidity state of the AMM, then any party ℓ may deposit $x_\ell > 0$ of X and $y_\ell > 0$ of Y in a proportion that reflects the current price: $\frac{x_\ell}{y_\ell} = \frac{x}{y}$.³ Then the state (x, y) is updated appropriately, we do $x_0 \leftarrow x$ and $y_0 \leftarrow y$, and finally, $L \leftarrow L \cup \{\ell\}$. The depositing LP receives shares in the usual way, but in addition it is allowed to specify its contribution $c_\ell \in [a, b]$ in the `aggregate()` algorithm (Equation 1), changing the state curve at the next epoch boundary. However, an LP can freeze its liquidity when it considers market conditions to be adverse. The LP establishes a bound on the spot price of tokens (Y) in terms of cash (X) below which its liquidity will not participate. Each liquidity provider $\ell \in L \cup L'$ sets a lower bound $d_\ell \geq 0$, updated at the beginning of each epoch (after `removeLiquidity()` and `liquidate()` defined subsequently). A liquidity provider ℓ can *alter* its d_ℓ at any time, but no change will occur until the next epoch boundary.

For any initial state (x_0, y_0) and current state (x, y) of the AMM with aggregate constant c , let p denote the (instantaneous) *spot price*

$$p = \frac{1}{-f'_{c, x_0, y_0}(x)},$$

as defined in Definition 5.4. This value is stored in the AMM and is updated appropriately. As soon as $p < d_\ell$ for some ℓ , the AMM moves ℓ 's liquidity into a distinct, untraded liquidity pool. If, at a later time, $p \geq d_\ell$, the AMM unfreezes ℓ 's liquidity by moving it back into the trading state. To ensure no consumer or producer pushes p to be much smaller than some d_ℓ , trades are restricted to *one unit increments*. The assumption is primarily made for simplification of the proof of Proposition 6.3. For completeness, we write the exact trading algorithm for multiple tokens Y and the corresponding system state changes in pseudocode below. A more efficient implementation is a discretized version of Uniswap V3's concentrated liquidity [13]. We delay discussion of Lines 13-14 and 27-28 to Section 7.

We showed in Proposition 5.7 that the price p is invariant under adding and removing liquidity, so we update p only on epoch boundaries and after every trade. The AMM maintains the set $\{d_\ell\}_{\ell \in L \cup L'}$ and a map $\mathbf{fr} : L' \rightarrow \mathbb{R}^2$ to keep track of the liquidity that inactive providers have the right to. Immediately after every epoch boundary and every trade, we execute the method `freeze_unfreeze()` defined in Algorithm 2.

³In an initial state where $x = 0$ or $y = 0$, there are no restrictions on how liquidity is added.

Algorithm 1: trade(tokenIn, amountIn, address)

```
1 if tokenIn = X then
2   amountUsed  $\leftarrow$  0
3   done  $\leftarrow$  false
4   while  $L \neq \emptyset$  and not done do
5     Let price be the current price in X to buy one Y token
6     amountUsed  $\leftarrow$  amountUsed + price
7     if amountUsed > amountIn then
8       done  $\leftarrow$  true
9     else
10      transferIn(price, X, source = address)
11      transferOut(1, Y, destination = address)
12      freeze_unfreeze()
13      for  $\ell \in L$  do
14         $r_\ell \leftarrow r_\ell + s_\ell \cdot \text{price}$ 
15 if tokenIn = Y then
16   tokensUsed  $\leftarrow$  0
17   done  $\leftarrow$  false
18   while  $L \neq \emptyset$  and not done do
19     Let price be the current price in X to sell one Y token
20     tokensUsed  $\leftarrow$  tokensUsed + 1
21     if tokensUsed > amountIn then
22       done  $\leftarrow$  true
23     else
24      transferIn(1, Y, source = address)
25      transferOut(price, X, destination = address)
26      freeze_unfreeze()
27      for  $\ell \in L$  do
28         $r_\ell \leftarrow r_\ell + s_\ell \cdot \text{price}$ 
```

Algorithm 2: freeze_unfreeze()

```
1 unfreeze()
2 freeze()
```

The `freeze()` function finds all active liquidity providers ℓ whose price tolerance d_ℓ is less than the current price, and moves their liquidity into a non-trading pool. If $L \neq \emptyset$, `unfreeze()` finds the inactive liquidity providers ℓ whose price tolerance d_ℓ is

at least p , and moves as much of their inactive liquidity back into the trading pool as possible, subject to preserving the ratio of x to y in the liquidity pool. If, however, $L = \emptyset$, the instantaneous price p is not well-defined, so we retrieve the values of x and y from the last point in time where some liquidity provider was active, and unfreeze all the liquidity providers to have a liquidity ratio of x/y . In both cases $L = \emptyset$ or $L \neq \emptyset$, the `freeze_unfreeze()` method freezes or unfreezes liquidity providers as needed.

Algorithm 3: freeze()

```

1  $L_{\text{freeze}} \leftarrow \{\ell \in L : p < d_\ell\}$ 
2  $L \leftarrow L \setminus L_{\text{freeze}}$ 
3  $L' \leftarrow L' \cup L_{\text{freeze}}$ 
4 for  $\ell \in L_{\text{freeze}}$  do
5    $\text{fr}(\ell) \leftarrow (s_\ell x, s_\ell y)$ 
6  $s_{\text{total}} \leftarrow \sum_{k \in L} s_k$ 
7  $x \leftarrow s_{\text{total}} x$ 
8  $y \leftarrow s_{\text{total}} y$ 
9 for  $\ell \in L$  do
10   $s_\ell \leftarrow \frac{s_\ell}{s_{\text{total}}}$ 

```

Algorithm 4: unfreeze()

```

1 if  $L = \emptyset$  then
2   Let  $r$  be the ratio  $x/y$  the last instant where  $L \neq \emptyset$ .
3   while  $\exists \ell \in L'$  do
4      $\text{unfreezeProvider}(\ell, r)$ 
5      $L \leftarrow L'$ 
6      $L' \leftarrow \emptyset$ 
7 else
8    $L_{\text{unfreeze}} \leftarrow \{\ell \in L' : p \geq d_\ell\}$ 
9   while  $\exists \ell \in L_{\text{unfreeze}} \cap L'$  do
10     $\text{unfreezeProvider}(\ell, x/y)$ 

```

Finally, we define `removeLiquidity()` and `liquidate()`. These methods are similar, but with one key difference. When `removeLiquidity()` is called by some $\ell \in L \cup L'$, the AMM adds ℓ to a set L_{remove} initialized to $\emptyset$; at the next epoch boundary, the AMM simply sends the assets that each $\ell \in L_{\text{remove}}$ has the right to (computed in the obvious way, depending on if $\ell \in L$ or $\ell \in L'$) back to ℓ , while

Algorithm 5: unfreezeProvider(ℓ, r)

```
1 require  $\ell \in L'$ 
2  $(x_\ell, y_\ell) \leftarrow \text{fr}(\ell)$ 
3 if  $x_\ell/y_\ell \geq r$  then
4    $x'_\ell \leftarrow ry_\ell$ 
5    $y'_\ell \leftarrow y_\ell$ 
6    $y \leftarrow y + y_\ell$ 
7    $x \leftarrow x + x'_\ell$ 
8    $\text{transferOut}(x_\ell - x'_\ell, X, \text{destination} = \ell)$ 
9 else
10   $x'_\ell \leftarrow x_\ell$ 
11   $y'_\ell \leftarrow x_\ell/r$ 
12   $y \leftarrow y + y'_\ell$ 
13   $x \leftarrow x + x_\ell$ 
14   $\text{transferOut}(y_\ell - y'_\ell, Y, \text{destination} = \ell)$ 
15  $L' \leftarrow L' \setminus \{\ell\}$ 
16  $L \leftarrow L \cup \{\ell\}$ 
17  $s_\ell \leftarrow \frac{x'_\ell}{x}$ 
18 for  $l \in L \setminus \{\ell\}$  do
19    $s_l \leftarrow s_l \cdot (1 - s_\ell)$ 
```

doing the appropriate bookkeeping. As this is the only difference from usual practice (e.g., [1]), so pseudocode is omitted.

Like `removeLiquidity()`, when `liquidate()` is called by an LP $\ell \in L \cup L'$, then ℓ is added to a set $L_{\text{liquidate}}$. At each epoch boundary, the following is executed in an atomic step: for each liquidity provider in $L_{\text{liquidate}}$, get the amounts of X and Y they have the right to, remove this liquidity, sell the total units of Y that have been traded for X , and distribute these funds to each member of $L_{\text{liquidate}}$ in proportion to their shares of asset X ; then set $L_{\text{liquidate}} \leftarrow \emptyset$. This protocol is listed as Algorithm 6. Both these methods run atomically at each epoch boundary, where `removeLiquidity()` is executed first.

6.1 Freezing, Removal, and Liquidation Properties

The advantage of including two different ways of removing liquidity and only at epoch boundaries is primarily to spread out the loss of the liquidity providers who choose to liquidate and provide those who call `removeLiquidity()` a guaranteed risk-free liquidity removal; the received liquidity from the `removeLiquidity()` method is not affected by liquidity providers calling `liquidate()` since the AMM's execution of `removeLiquidity()` precedes that of `liquidate()`.

$\mathbb{R}_{>0}^2$ satisfy $\frac{x'}{y'} = \frac{x''}{y''}$ and $x'' < x'$ and $y'' < y'$, then

$$|f'_{c,x'',y''}(x)| \geq |f'_{c,x_0,y_0}(x + x' - x'')|,$$

for all $x \in (0, x'']$.

Proof. Suppose $(x', y') \in \text{state}(f_{c,x_0,y_0})$ and $(x'', y'') \in \mathbb{R}_{>0}^2$ satisfy $\frac{x'}{y'} = \frac{x''}{y''}$ and $x'' < x'$ and $y'' < y'$. Notice that

$$f'_{c,x_0,y_0}(x) = -cy_0 \left(\frac{x}{x_0} \right)^{-c-1} \cdot \frac{1}{x_0} = -c \cdot \frac{f_{c,x_0,y_0}(x)}{x},$$

and similarly,

$$f'_{c,x'',y''}(x) = -c \cdot \frac{f_{c,x'',y''}(x)}{x}.$$

Since $(x', y') \in \text{state}(f_{c,x_0,y_0})$, we know

$$f_{c,x_0,y_0}(x) = y_0 x_0^c x^{-c} = y_0 \left(\frac{x'}{x_0} \right)^{-c} (x')^c x^{-c} = f_{c,x_0,y_0}(x') \cdot (x')^c x^{-c} = y'(x')^c x^{-c}.$$

Fix some $x \in (0, x'']$. Since $x' - x'' \geq 0$, $0 < x'' \leq x'$, and $\frac{x'}{x''} = \frac{y'}{y''}$, we have

$$\begin{aligned} \left(\frac{x + x' - x''}{x} \right)^{c+1} &\geq \left(1 + \frac{x' - x''}{x} \right)^{c+1} \geq \left(1 + \frac{x' - x''}{x''} \right)^{c+1} \\ &= \left(\frac{x'}{x''} \right)^{c+1} = \frac{y'}{y''} \cdot \left(\frac{x'}{x''} \right)^c. \end{aligned}$$

It follows that

$$y'' \cdot x^{-c-1} (x'')^c \geq y'(x + x' - x'')^{-c-1} (x')^c.$$

Multiplying by c yields that

$$\begin{aligned} |f'_{c,x'',y''}(x)| &= c \cdot \frac{f_{c,x'',y''}(x)}{x} = c \cdot y'' x^{-c-1} (x'')^c \\ &\geq c \cdot \frac{y' \left(\frac{x+x'-x''}{x'} \right)^{-c}}{x + x' - x''} = c \cdot \frac{f_{c,x_0,y_0}(x + x' - x'')}{x + x' - x''} \\ &= |f'_{c,x_0,y_0}(x + x' - x'')|. \end{aligned}$$

□

Lemma 6.2. Suppose some party sells one unit of Y to the AMM for x_1 of X , and then sells another unit of Y to the AMM for x_2 of X . Then $x_1 \geq x_2$.

Proof. Let f_{c,x_0,y_0} be the state curve before the first trade. If L and L' are the same after the first trade (so `freeze_unfreeze()` did not update any global variables), then since f_{c,x_0,y_0} is a convex function, we certainly have $x_1 \geq x_2$.

Now suppose the execution of `freeze_unfreeze()` changed the sets L and L' after the first trade. It suffices to prove that x_2 is at most the amount of X one would get if the state curve never updated and was always equal to f_{c,x_0,y_0} , given the observation above. To this end, we consider two possible cases for the second trade. Let $(x', y') \in \text{state}(f_{c,x_0,y_0})$ be the state after the first trade but before calling `freeze_unfreeze()`; let (x'', y'') be the state after completing the first trade, so the state curve updates to $f_{c,x'',y''}$. Observe that the updated set L after the first trade is a subset of the initial L because the instantaneous price p can only decrease after the first selling trade. By inspection of `freeze()` and `unfreeze()`, we see that $\frac{x'}{y'} = \frac{x''}{y''}$ but $0 < x'' < x'$ and $0 < y'' < y'$.

Let x'_2 be the amount received for the second trade if the state curve is f_{c,x_0,y_0} ; let x''_2 be the amount received for the second trade if the state curve is $f_{c,x'',y''}$. The convexity of f_{c,x_0,y_0} implies that $x_1 \geq x'_2$. If we can show $x'_2 \geq x''_2$, then we know $x_1 \geq x_2$ because $x_2 \in \{x'_2, x''_2\}$.

By Lemma 6.1, $|f'_{c,x'',y''}(x)| \geq |f'_{c,x_0,y_0}(x + x' - x'')|$ for all $x \in (0, x'']$. This implies that

$$\left| \frac{d}{dy} f_{c,x'',y''}^{-1}(y) \right| \leq \left| \frac{d}{dy} f_{c,x_0,y_0}^{-1}(y + y' - y'') \right|$$

for all $y \in [y'', \infty)$.

Finally, we prove $x'_2 \geq x''_2$, showing the lemma. We have

$$\begin{aligned} x'_2 &= f_{c,x_0,y_0}^{-1}(y') - f_{c,x_0,y_0}^{-1}(y' + 1) \\ &= \int_{y'+1}^{y'} \frac{d}{dy} f_{c,x_0,y_0}^{-1}(y + y' - y'') dy \\ &= \int_{y''}^{y''+1} \left| \frac{d}{dy} f_{c,x_0,y_0}^{-1}(y + y' - y'') \right| dy \\ &\geq \int_{y''}^{y''+1} \left| \frac{d}{dy} f_{c,x'',y''}^{-1}(y) \right| dy \\ &= \int_{y''+1}^{y''} \frac{d}{dy} f_{c,x'',y''}^{-1}(y) dy \\ &= f_{c,x'',y''}^{-1}(y'') - f_{c,x'',y''}^{-1}(y'' + 1) = x''_2. \end{aligned}$$

□

Proposition 6.3. *Let $\ell \in L \cup L'$. Suppose that in some execution α , the liquidity provider ℓ calls `removeLiquidity()` to obtain $x_{\ell,1}^\alpha$ of X and $y_{\ell,1}^\alpha$ of Y , and then trades $y_{\ell,1}^\alpha$ of Y to the AMM in exchange for $x_{\ell,2}^\alpha$ of X , so that ℓ receives $x_\ell^\alpha = x_{\ell,1}^\alpha + x_{\ell,2}^\alpha$ of X . Suppose that in some otherwise identical execution β , ℓ instead calls `liquidate()` and receives x_ℓ^β of X . Then $x_\ell^\alpha \leq x_\ell^\beta$.*

Proof. Consider the sets $L_{\text{liquidate}}^\alpha$ and $L_{\text{liquidate}}^\beta$ in executions α and β , respectively. Consider the c , which is the same in both executions. Notice that, by construction, the effective AMM state (x_0, y_0) after executing the loop in line 14 of `liquidate()` is the same in both executions α and β . Suppose that in execution α , there is some other liquidity provider (not ℓ) that removed liquidity and sold it to the system before ℓ . By Lemma 6.2, the received amount $x_{\ell,2}^\alpha$ of X is at most the amount ℓ would have received if ℓ was the first provider to sell their removed liquidity back to the AMM. Thus it suffices to prove the claim when ℓ was the first liquidity provider to sell their liquidity back to the AMM after `liquidate()` was called in execution α .

To this end, consider the sequence $(x_k^\alpha)_{k=1}^n$ of received amounts of X for consecutive single-unit trades made by `liquidate()` and ℓ 's atomic selling trades, in execution α . Similarly, let $(x_k^\beta)_{k=1}^n$ be the sequence of received amounts of X for consecutive single-unit trades made by `liquidate()`, in execution β . It is clear that $n \leq y_{\text{liquidate}}$, as defined in `liquidate()`, so that we may extend both sequences by setting $x_k^\alpha = x_k^\beta = 0$ for $n < k \leq y_{\text{liquidate}}$. By the assumption made above, these two sequences are equal: $x_k^\alpha = x_k^\beta$ for all $k \in [y_{\text{liquidate}}]$. Then we may let $(x_k)_{k=1}^{y_{\text{liquidate}}}$ be this sequence. By Lemma 6.2, $(x_k)_{k=1}^{y_{\text{liquidate}}}$ is a nonincreasing sequence. Thus

$$x_{\ell,2}^\alpha = \sum_{k=y_{\text{liquidate}}-y_{\ell,1}^\alpha+1}^{y_{\text{liquidate}}} x_k^\alpha \leq \frac{y_{\ell,1}^\alpha}{y_{\text{liquidate}}} \cdot \sum_{k=1}^{y_{\text{liquidate}}} x_k = \overline{s}_\ell \cdot x_{\text{liquidate}},$$

where $\overline{s}_\ell$ and $x_{\text{liquidate}}$ are defined in `liquidate()`. Note that the x_ℓ in `liquidate()` is equal to $x_{\ell,1}^\alpha$, so that

$$x_\ell^\alpha = x_{\ell,1}^\alpha + x_{\ell,2}^\alpha \leq x_{\ell,1}^\alpha + \overline{s}_\ell \cdot x_{\text{liquidate}} = x_\ell^\beta.$$

□

We also show that the `freeze_unfreeze()` method yields a *stable* liquidity state; that is, a liquidity provider $\ell \in L \cup L'$ is active if and only if $p \geq d_\ell$. Proving this is important for proving LP Stability in Theorem 8.2.

Proposition 6.4. *Consider the AMM state immediately after calling `freeze_unfreeze()`. Let $\ell \in L \cup L'$. Then $\ell \in L$ if and only if $p \geq d_\ell$.*

Proof. Inspecting both `freeze()` and `unfreeze()`, the AMM state changes of the variables (x, y) are indistinguishable from a sequence of adding and removing liquidity. By Lemma 5.7, the current instantaneous price p is invariant throughout `freeze_unfreeze()`. If $L = \emptyset$ before executing `freeze_unfreeze()`, then this implies that after executing `unfreeze()` and `freeze()`, we can see that $\ell \in L$ if and only if $\ell \notin L_{\text{freeze}}$ (as defined in `freeze()`), which holds if and only if $p \geq d_\ell$. On the other hand, if $L \neq \emptyset$ before executing `freeze_unfreeze()`, then by a similar reasoning, $\ell \in L$ after executing the algorithm if and only if $p \geq d_\ell$. □

7 Market Clearing Auction

In this section, we describe a mechanism for bargain hunters, consumers with low price and low urgency willing to wait until the last minute to acquire tokens. We also describe a complementary mechanism for producers to ensure market clearance.

The first protocol we use is a form of a second-price auction. Importantly, we fairly distribute these profits to LPs according to their cumulative participation in the protocol before retrieval time t_r . Informally, if an LP ℓ takes on more risk by participating in a large number of trades with a large share of liquidity, then, in addition to receiving more fees, the market clearing auction will prioritize clearing ℓ 's remaining tokens (Y) for cash (X) at a higher price than other, more risk-averse, LPs.

A *consumer order*, or *bid* b has two attributes:

- **address**(b): if a party u submits the order b , we set **address**(b) = u .
- **price**(b): the party **address**(b) sets a price **price**(b) > 0 that they are willing to pay for one unit of the token.

When a bid is submitted, it is not locked; any user can remove any of their bids if desired, at any time. For ease of exposition, each bid is for a single unit of Y . A user who wishes to buy multiple units of Y simply submits multiple bids.

The AMM maintains a dynamic list $\mathcal{B}$ that keeps track of all existing bids at any given time. The list $\mathcal{B}$ is called the *consumer order book*. Initially $\mathcal{B}$ is empty, and every time a bid b is submitted, b is added to $\mathcal{B}$, so that $\mathcal{B}$ is monotone decreasing by the price of each bid (highest price to lowest price). This implies that $\mathcal{B}[0]$ is a bid with maximum price. If the AMM price p_Y for a single unit of Y falls to below **price**(b) for the bid $b = \mathcal{B}[0]$, then b is removed from $\mathcal{B}$ and one unit of Y is immediately sold to **address**(b) via a trade at the price $p_Y \leq \mathbf{price}(b)$, so that the profits and fees of the trade are distributed to the active liquidity providers (L) in proportion to their shares as usual.

We next describe the market clearing algorithm **clearing**($\cdot$). From the perspective of a buyer, **clearing**($\cdot$) is purely a second-price auction. However, the distribution of profits to the LPs is more intricate. Each LP $\ell \in L \cup L'$ is assigned some variable r_ℓ that informally keeps track of the cumulative risk ℓ has faced.⁴ When a trade (buying or selling) that exchanges x' of X and some amount of Y , we do the following:

- For each $\ell \in L$, compute $x_\ell \leftarrow s_\ell x'$ and increment $r_\ell \leftarrow r_\ell + x_\ell$.⁵

Now consider the state of the AMM at the retrieval time t_r . We execute the following market clearing algorithm **clearing**($\cdot$) at this time.

⁴ r_ℓ is initialized to 0 when ℓ joins the set of LPs.

⁵Fees are also distributed in proportion to this x_ℓ .

Algorithm 7: clearing()

```

1 for  $\ell \in L$  do
2   | Global :  $x_\ell \leftarrow s_\ell x$ 
3   | Global :  $y_\ell \leftarrow s_\ell y$ 
4 for  $\ell \in L'$  do
5   | Global :  $(x_\ell, y_\ell) \leftarrow \text{fr}(\ell)$ 
6 for  $\ell \in L \cup L'$  do
7   | Global :  $\overline{s}_\ell \leftarrow \frac{r_\ell}{\sum_{k \in L \cup L'} r_k}$ 
8 Global :  $L_{\text{available}} \leftarrow L \cup L'$ 
9 while  $|\mathcal{B}| \geq 1$  and  $\sum_{\ell \in L \cup L'} y_\ell \geq 1$  do
10  |  $b \leftarrow \mathcal{B}[0]$ 
11  | delete  $\mathcal{B}[0]$ 
12  |  $p \leftarrow \text{price}(\mathcal{B}[0])$  /* Second price */
13  | execute( $b, p$ )
14 for  $\ell \in L \cup L'$  do
15  | transferOut( $x_\ell, X, \text{destination} = \ell$ )
16  | transferOut( $y_\ell, Y, \text{destination} = \ell$ )

```

Essentially, we execute the bid b with highest price for the second-highest price in $\mathcal{B}$, and the profits and fees from this trade are distributed so that the liquidity providers ℓ get priority proportional to their cumulative risk variable r_ℓ . If all LPs $\ell \in L \cup L'$ have sufficiently many tokens, in particular, $y_\ell \geq \overline{s}_\ell$, then the profits to ℓ are distributed in exact proportion to r_ℓ . Otherwise, we keep running a similar process until the system can “complete” a unit of Y , keeping profits to LPs in proportion to r_ℓ . This provides an incentive for liquidity providers to keep the system *live*: to not leave the AMM early, and to keep providing ample liquidity until the retrieval time.

In addition to maintaining the consumer order book, the AMM keeps track of a *producer order book* $\mathcal{P}$, so that any party that has a unit of Y can escrow it into the system, and sell it for some minimum price that they set. In particular, a *producer order* s has two attributes:

- **address**(s): if a party u submits the selling order s , we set **address**(s) = u .
- **price**(s): the party **address**(s) sets a price **price**(s) > 0 for which they are willing to sell one unit of the token.

The producer order book $\mathcal{P}$ is a list, initialized to $\emptyset$, that is always sorted from least to greatest (the reverse of $\mathcal{B}$) by the price of selling orders. At any time, a user can submit a producer order s by escrowing one unit of Y into the AMM and setting a minimum selling price **price**(s) for that producer order. If the AMM price q_Y to

Algorithm 8: $\text{execute}(b, p)$

```
1  $y_{\text{total}} \leftarrow 0$ 
2 while  $y_{\text{total}} < 1$  do
3    $L_{\text{toRemove}} \leftarrow \emptyset$ 
4   for  $\ell \in L_{\text{available}}$  do
5     if  $y_\ell < \bar{s}_\ell(1 - y_{\text{total}})$  then
6        $y_{\text{total}} \leftarrow y_{\text{total}} + y_\ell$ 
7        $y_\ell \leftarrow 0$ 
8        $x_\ell \leftarrow x_\ell + p \cdot y_\ell$ 
9        $L_{\text{toRemove}} \leftarrow L_{\text{toRemove}} \cup \{\ell\}$ 
10      for  $k \in L_{\text{available}} \setminus L_{\text{toRemove}}$  do
11         $\bar{s}_k \leftarrow \frac{s_k}{1 - \bar{s}_\ell}$ 
12      else
13         $y_{\text{total}} \leftarrow y_{\text{total}} + \bar{s}_\ell(1 - y_{\text{total}})$ 
14         $y_\ell \leftarrow y_\ell - \bar{s}_\ell(1 - y_{\text{total}})$ 
15         $x_\ell \leftarrow x_\ell + p \cdot \bar{s}_\ell(1 - y_{\text{total}})$ 
16     $L_{\text{available}} \leftarrow L_{\text{available}} \setminus L_{\text{toRemove}}$ 
17  $\text{transferOut}(1, Y, \text{destination} = \text{address}(b))$ 
18  $\text{transferIn}(p, X, \text{source} = \text{address}(b))$ 
```

sell one unit of Y ever rises so that $s = \mathcal{P}[0]$, the producer order with the lowest price, has a price $\text{price}(s) \leq q_Y$, then s is removed from $\mathcal{P}$ and one unit of Y is immediately sold to the AMM from $\text{address}(s)$ via a regular AMM trade at the price q_Y .

We also impose a standard kind of interaction between $\mathcal{B}$ and $\mathcal{P}$ that functions identically to a limit order book. Any time a new consumer or producer order is submitted to the AMM, we run the following algorithm. While $|\mathcal{B}| > 0$ and $|\mathcal{P}| > 0$ and $\text{price}(\mathcal{B}[0]) \geq \text{price}(\mathcal{P}[0])$, then do $b \leftarrow \mathcal{B}[0]$, $s \leftarrow \mathcal{P}[0]$; do

```
transferOut(1, Y, destination = address(b));
transferOut(price(b), X, destination = address(s));
```

finally, remove b from $\mathcal{B}$ and remove s from $\mathcal{P}$. No state variables are updated, since the funds for the orders are initially escrowed from the addresses of the orders.

This order-book style protocol is only used in extreme market conditions: (i) all but one unit of Y remains in the liquidity pool, so that no consumer can make a trade with the AMM state curve; or (ii) all liquidity providers have been frozen, so that, again, no more trades can happen. In these two scenarios, the consumers and producers must submit consumer orders and producer orders, so that the interaction between the consumer order book and producer order book allows the market to

remain live until the retrieval time t_r .

8 Liveness and Market Clearance

Two of the most important properties that a DEX can satisfy are *liveness* and *market clearance*. The liveness property in our context states that at any time before the retrieval time t_r , (a) there are always actions that parties can take (via the DEX) to execute a trade (trades are always possible), and (b) if the time is less than $\max T$, then all liquidity providers ℓ can take some action so that $\ell \in L$ after the next time in T (liquidity providers are not permanently locked).

In addition to liveness, *market clearance* is a desirable property of a DEX, and mirrors the safety condition usually desired in a distributed system. In this context, *market clearance* means that (a) liquidity providers are always in a *stable* state (in the context of Proposition 6.4), (b) no two parties that are willing to trade do not end up trading by retrieval time, and (c) if the DEX contains any tokens left in the system after executing the clearing protocol `clearing()`, then there is no remaining consumer. Just as *safety* in a distributed system requires that no correct processors ever have inconsistent views, market clearance in our AMM requires that there are no inconsistencies between the actions that parties wish to take and the transactions that actually happen in the DEX.

Now, we formally define and prove the liveness of our DEX model. Let `DynamicAMM` be the DEX described in this paper. We assume that there are three types of parties: buyers (consumers), sellers (producers), and liquidity providers.

Definition 8.1. *The DEX `DynamicAMM` is live if the following hold:*

- (Consumer Liveness) *Consider any time $t < t_r$, a buyer u_b at time t , and a seller u_s at time t . Then there is a sequence of transactions that can be submitted and executed by a nonempty subset of the parties $\{u_b, u_s\}$ where u_s sells a Y token or u_b buys a Y token.*
- (LP Liveness) *Given any liquidity provider ℓ , a time $t < \max T$, and the AMM state at time t , there is a transaction that can be submitted by ℓ so that $\ell \in L$ when the time is at least $\min\{t' \in T : t' > t\}$ and at most t_r .*

Theorem 8.1 (Liveness). *The DEX `DynamicAMM` is live.*

Proof. Consider any time $t < t_r$, a buyer u_b , and a seller u_s . Regardless of if a trade can be made using the AMM curve of `DynamicAMM` or not, u_b and u_s can submit the following transactions. First, u_b submits a consumer order b for a price q ; then u_s submits a producer order s for the same price q . If either of these two orders can be executed using the AMM curve, then at least one of them will be, so that we are done.

Otherwise, the interaction between $\mathcal{B}$ and $\mathcal{P}$ described in Section 7, implies that at least one of the orders b and s will be executed, showing that the AMM satisfies Consumer Liveness. (This still holds even if some other order is submitted between when the orders b and s are submitted, or if some other orders are removed.)

Let ℓ be a liquidity provider at time $t < \max T$. Then ℓ can simply request that $d_\ell \leftarrow 0$ at time t . Then **DynamicAMM** will update $d_\ell \leftarrow 0$ at the time $t' = \min\{t' \in T : t' > t\}$ and then immediately run **freeze_unfreeze()**. By inspection, $\ell \in L$ after running **unfreeze()**. Since $p > 0$ and $d_\ell = 0$ at time t' , we have $p \geq d_\ell$, so that $\ell \notin L_{\text{freeze}}$ in the **freeze()** algorithm; hence $\ell \in L$ after executing **freeze_unfreeze()**. At any time in (t', t_r) , we will still have $p > 0$, so that $\ell \in L$. Thus **DynamicAMM** also satisfies LP Liveness. $\square$

Let us now formally define and prove market clearance in the context of our DEX. We assume that participants of each type are rational, as defined below:

- Each liquidity provider ℓ has a sequence of prices⁶ $(q_t)_{t \in T} = (q(\ell)_t)_{t \in T}$ where they are willing to provide liquidity to **DynamicAMM** and engage in a trade as long as the current spot price is at least q_t , where t is the greatest time in T less than the current time. In this case, we assume that each $\ell \in L \cup L'$ has sets $d_\ell \leftarrow q_t$ at each time $t \in T$. Furthermore, if the current time t is in $(\max T, t_r)$, then every remaining liquidity provider is willing to sell the remaining tokens of Y for any price when the time is in $(\max T, t_r)$; that is, $q_{\max T} = 0$.
- For each buyer u_b , there is some fixed price $q = q(u_b)$ such that if the AMM price to buy one unit of Y falls to at most q , then u_b will buy at the AMM price; if the producer order $s = \mathcal{P}[0]$ satisfies **price**(s) $\leq q$, then u_b will buy from the seller **address**(s) for the price **price**(s) (by issuing a consumer order for a price **price**(s)); if both of these happen at the same time, u_b will buy the token Y at the lower price of the two available options. At some time $t < t_r$, if u_b has not bought a Y token by time t , they will register a bid for a price q .
- For each seller u_s , there is some fixed price $q = q(u_s)$ such that if the AMM price to sell one unit of Y rises to at least q , then u_s will sell at the AMM price; if the bid $b = \mathcal{B}[0]$ with **price**(b) $\geq q$, then u_s will sell to the buyer **address**(b) for the price **price**(b); if both of these happen at the same time, u_s sells the token Y at the higher price of the two available options. At some time $t < t_r$, if u_s has not sold a Y token by time t , they will register a producer order for a price q .

Definition 8.2. *We say that **DynamicAMM** satisfies market clearance if the following hold:*

⁶ ℓ discovers this sequence as time progresses and is not known a priori.

- (LP Stability) *At all times t between the initialization of the AMM and retrieval time for which $L \neq \emptyset$, a liquidity provider $\ell \in L \cup L'$ is active (in L) if and only if $q(\ell)_{t_0} \geq p$, where t_0 is the greatest time in T less than t .*
- (Consumer Clearance) *At time t_r , there is no buyer u_b and seller u_s where u_b is willing to buy at a price that is at least what u_s is willing to sell for.*
- (System Clearance) *At time t_r , after `clearing()` has executed, if `DynamicAMM` contains any of Y , then there is no remaining buyer.*

Theorem 8.2. *The DEX `DynamicAMM` satisfies market clearance.*

Proof. Consider the state of `DynamicAMM` at an arbitrary time t for which $L \neq \emptyset$. Let $t_0 = \max\{t' \in T : t' < t\}$. Let $\ell \in L \cup L'$. Then by assumption, at time t_0 , ℓ must have set $d_\ell \leftarrow q(\ell)_{t_0}$. Notice that the value of p at time t is the same as it was at the latest time $t_1 < t$ for which `freeze_unfreeze()` was called, since `freeze_unfreeze()` is called precisely when p changes. Then Proposition 6.4 implies that at time t_1 , we know $\ell \in L$ if and only if $q(\ell)_{t_0} = d_\ell \geq p$ at time t_1 . Since p is the same at time t as it was at time t_1 , we know that $\ell \in L$ if and only if $q(\ell)_{t_0} \geq p$ at time t . Thus `DynamicAMM` satisfies LP Stability.

For the remainder of this proof, consider the state of `DynamicAMM` at time t_r . Suppose there is a buyer u_b and a seller u_s where $q(u_b) \geq q(u_s)$. Since u_b and u_s are unfulfilled at retrieval time, our hypotheses on buyers and sellers imply that there is a bid $b \in \mathcal{B}$ and producer order $s \in \mathcal{P}$ such that `address`(b) = u_b , `address`(s) = u_s , `price`(b) = $q(u_b)$, `price`(s) = $q(u_s)$. If s was submitted to `DynamicAMM` before b , then the submission of b (which was before t_r) succeeds the check `price`($\mathcal{P}[0]$) > `price`(b) (otherwise u_b would not be unfulfilled); because $\mathcal{P}$ is sorted in increasing order by price, `price`(s) $\geq$ `price`($\mathcal{P}[0]$) > `price`(b), so that $q(u_s) > q(u_b)$, a contradiction. The case when b was submitted before s gives a similar contradiction. Hence `DynamicAMM` satisfies Consumer Clearance.

Finally, suppose that `DynamicAMM` contains at least one unit of Y after `clearing()` executed, and that there is some unfulfilled buyer u_b . Clearly `clearing()` does eventually terminate, and $\sum_{\ell \in L \cup L'} y_\ell > 0$ when `clearing()` terminates. The termination criteria for `clearing()` imply that $\mathcal{B} = \emptyset$ after `clearing()` terminates. By our initial assumptions, u_b must have submitted a consumer order for the price $q(u_b)$ before the time t_r , so that before `clearing()` was executed, there was some $b \in \mathcal{B}$ with `address`(b) = u_b . We assumed that u_b is a remaining buyer after `clearing()` was executed, which implies $b \in \mathcal{B}$ upon termination, a contradiction. Hence `DynamicAMM` satisfies System Clearance. $\square$

9 Conclusion

This paper proposed novel DEX for trading assets that expire. This DEX includes both AMM and auction mechanisms: an AMM for normal execution, when expiration

is far away, and auctions when expiration is imminent. To address market failures that may accompany expiration, the DEX include an LP-controlled price adjustment protocol, and a hybrid market clearing protocol that combines the benefits of AMMs and incentive-compatible auctions.

Allowing LPs to dynamically adjust prices requires admitting a larger family of state curves than the constant-product curves commonly used by popular AMMs [1, 13]. LPs can intervene to adjust spot prices, where an LP’s influence is proportional to its investment.

As expiration approaches, prices may become highly volatile. To discourage LPs from withdrawing their investments, the DEX allows nervous LPs to freeze their assets if spot prices fall below a specified minimum, and unfreeze if it later rises above that minimum.

The DEX includes a market clearing mechanism that functions in two ways. First, it can function as a limit order book before the expiration date to ensure market-liveness when all the liquidity providers are frozen, or when no more tokens can be bought from the system at a certain time. Second, it acts like a second-price auction to sell the remaining tokens in the system to unfulfilled customers. One key difference between our market clearing protocol and a standard second price auction is the way profits are distributed to liquidity providers: We distribute profits in proportion to the total participation of the liquidity providers, measured in the sum of the absolute change of the LP’s cash over all trades.

We prove a number of properties which show our DEX construction is sound. First, we prove some soundness properties of our price and curve adjustment protocol – such as monotonicity of instantaneous price as a function of c and price-invariance under dynamic liquidity. Second, we prove a bound on the profit obtained by liquidity providers when given the option of calling two different, intricately connected methods of removal; this shows that it is always best to liquidate and share the losses with everyone else who wishes to liquidate, instead of attempting to act selfishly. Finally, we prove that our system is live – which roughly means that it cannot be permanently frozen until retrieval time – and it clears the market – there are no two remaining parties who would have been willing to make a trade but did not meet through the system.

References

- [1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. <https://uniswap.org/whitepaper.pdf>, March 2020. As of 8 February 2021.
- [2] Guillermo Angeris and Tarun Chitra. Improved Price Oracles: Constant Function Market Makers. *SSRN Electronic Journal*, 2020.

- [3] Guillermo Angeris, Alex Evans, and Tarun Chitra. When does the tail wag the dog? Curvature and market making. *arXiv:2012.08040 [q-fin]*, December 2020. arXiv: 2012.08040.
- [4] Guillermo Angeris, Hsien-Tang Kao, Rei Chiang, Charlie Noyes, and Tarun Chitra. An analysis of Uniswap markets. *arXiv:1911.03380 [cs, math, q-fin]*, February 2021. arXiv: 1911.03380.
- [5] Jun Aoyagi. Lazy Liquidity in Automated Market Making. *SSRN Electronic Journal*, 2020.
- [6] Bancor. Proposing bancor v2.1: Single-sided amm with elastic bnt supply. <https://blog.bancor.network/proposing-bancor-v2-1-single-sided-amm-with-elastic-bnt>, October 2020. As of 8 February 2021.
- [7] Massimo Bartoletti, James Hsin-yu Chiang, and Alberto Lluch-Lafuente. A theory of Automated Market Makers in DeFi. *arXiv:2102.11350 [cs]*, April 2021. arXiv: 2102.11350.
- [8] Maxim Bichuch and Zachary Feinstein. Axioms for automated market makers: A mathematical framework in fintech and decentralized finance, 2023.
- [9] Agostino Capponi and Ruizhe Jia. The adoption of blockchain-based decentralized exchanges. *arXiv preprint arXiv:2103.08842*, 2021.
- [10] Zdravko Cvetkovski. *Inequalities: theorems, techniques and selected problems*. Springer Science & Business Media, 2012.
- [11] Michael Egorov. Stableswap - efficient mechanism for stablecoin liquidity. <https://www.curve.fi/stableswap-paper.pdf>, November 2019. As of 8 February 2021.
- [12] Daniel Engel and Maurice Herlihy. Composing Networks of Automated Market Makers. *ArXiv*, June 2021. arXiv: 2106.00083.
- [13] Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 Core, March 2021.
- [14] Eyal Hertzog, Guy Benartzi, and Galia Benartzi. Bancor protocol, 2017.
- [15] Fernando Martinelli and Nikolai Mushegian. Balancer: A non-custodial portfolio manager, liquidity provider, and price sensor. <https://balancer.finance/whitepaper/>, 2109. As of 2 February 2021.
- [16] Jason Milionis, Ciamac C. Moallemi, Tim Roughgarden, and Anthony Lee Zhang. Automated Market Making and Loss-Versus-Rebalancing, September 2022. arXiv:2208.06046 [math, q-fin].

- [17] Geoffrey Ramseyer, Mohak Goyal, Ashish Goel, and David Mazières. Augmenting batch exchanges with constant function market makers, 2023.
- [18] Jiahua Xu, Krzysztof Paruch, Simon Cousaert, and Yebo Feng. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Computing Surveys*, 55(11):1–50, 2023.
- [19] Yi Zhang, Xiaohong Chen, and Daejun Park. Formal specification of constant product ($x \cdot y = k$) market maker model and implementation. <https://github.com/runtimeverification/verified-smart-contracts/blob/uniswap/uniswap.v>, 2018.

A Aggregation Mechanics

Here, we will show that the weighted geometric mean aggregation algorithm described by equation 1 is the unique valid aggregation algorithm. To this end, fix an arbitrary finite nonempty set of LPs L , and let

$$\text{aggregate}_0((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = \prod_{\ell \in L} c_\ell^{s_\ell}.$$

Recall the definition of a *valid* aggregation algorithm, which we rewrite here for convenience.

Definition A.1. *Let $\text{aggregate}()$ be an aggregation algorithm; let L be the particular set of active LPs at some time, and consider the corresponding aggregation function. We say that $\text{aggregate}()$ is valid if the following axioms hold.*

1. *For each $\ell \in L$, aggregate has continuous partial derivatives with respect to c_ℓ . Additionally,*

$$\frac{\partial(\text{aggregate})}{\partial \log(c_\ell)} = s_\ell \cdot \text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}).$$

2. *If there exists some x such that $c_\ell = x$ for all $\ell \in L$, then*

$$\text{aggregate}((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = x.$$

Proposition A.1. *$\text{aggregate}_0()$ defines a valid aggregation algorithm.*

Proof. Fix some $\ell \in L$. We write $c = \text{aggregate}_0((c_k)_{k \in L}, (s_k)_{k \in L})$. Observe that

$$\begin{aligned}
\frac{\partial(\text{aggregate}_0)}{\partial \log c_\ell} &= \frac{\frac{\partial(\text{aggregate}_0)}{\partial c_\ell}}{\frac{\partial \log c_\ell}{\partial c_\ell}} \\
&= \frac{\frac{\partial}{\partial c_\ell} \prod_{k \in L} c_k^{s_k}}{\frac{1}{c_\ell}} \\
&= c_\ell s_\ell \cdot c_\ell^{s_\ell - 1} \cdot \prod_{k \in L \setminus \{\ell\}} c_k^{s_k} \\
&= s_\ell \prod_{k \in L} c_k^{s_k} = s_\ell c.
\end{aligned}$$

This shows condition 1 of Definition A.1. Finally, consider any $((c_k)_{k \in L}, (s_k)_{k \in L})$ in the domain of aggregate_0 such that there exists some x such that $c_\ell = x$ for all $\ell \in L$. Since $\sum_{k \in L} s_k = 1$,

$$c = \prod_{k \in L} c_k^{s_k} = \prod_{k \in L} x^{s_k} = x,$$

as desired. □

Now we prove the uniqueness of $\text{aggregate}_0()$.

Proposition A.2. $\text{aggregate}_0()$ is the only valid aggregation algorithm.

Proof. Let $\text{aggregate}()$ be any valid aggregation algorithm. It suffices to show that $\text{aggregate} = \text{aggregate}_0$ as functions, for all nonempty finite sets L . To this end, let L be finite and nonempty. Consider the function of the algorithm $\text{aggregate}()$ on LP set L , which we call c for brevity. Fix $(s_\ell)_{\ell \in L} \in [0, 1]^L$ such that $\sum_{\ell \in L} s_\ell = 1$. Since aggregate is valid, for all $\ell \in L$,

$$\frac{\partial c}{\partial \log(c_\ell)} = s_\ell \cdot c.$$

By the chain rule,

$$\begin{aligned}
\frac{\partial c}{\partial \log c_\ell} &= \frac{\frac{\partial c}{\partial \log c_\ell}}{\frac{\partial \log c_\ell}{\partial c_\ell}} \\
&= c_\ell \cdot \frac{\partial c}{\partial c_\ell}.
\end{aligned}$$

Thus

$$\begin{aligned}
\frac{\partial c}{\partial c_\ell} &= \frac{s_\ell}{c_\ell} \cdot c \\
\implies \int \frac{dc}{c} &= \int s_\ell \cdot \frac{dc_\ell}{c_\ell} \\
\implies \log(c) &= s_\ell \log(c_\ell) + A((c_k)_{k \in L \setminus \{\ell\}}) \\
&= \log(c_\ell^{s_\ell}) + A((c_k)_{k \in L \setminus \{\ell\}}) \\
\implies c &= c_\ell^{s_\ell} \cdot B((c_k)_{k \in L \setminus \{\ell\}})
\end{aligned}$$

for some functions $A((c_k)_{k \in L \setminus \{\ell\}})$ and $B((c_k)_{k \in L \setminus \{\ell\}})$. This implies that there exists a constant C (possibly dependent on $(s_\ell)_{\ell \in L}$) such that

$$c = C \cdot \prod_{\ell \in L} c_\ell^{s_\ell}.$$

Since this holds for any $(c_\ell)_{\ell \in L}$, we may choose $(c_\ell)_{\ell \in L}$ so that $c_\ell = x$ for all $\ell \in L$, for a fixed $x > 0$. Using the second condition of Definition A.1, we know that

$$c((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = x.$$

This implies

$$\begin{aligned}
x &= c((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) \\
&= C \cdot \prod_{\ell \in L} c_\ell^{s_\ell} \\
&= C \cdot \prod_{\ell \in L} x^{s_\ell} \\
&= C \cdot x
\end{aligned}$$

Since $x > 0$, it follows that $C = 1$. Thus, for any $((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L})$ in the domain of c , it follows that

$$c((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = \prod_{\ell \in L} c_\ell^{s_\ell},$$

as desired. □

We conclude by proving some basic properties of $\text{aggregate}_0()$.

Proposition A.3. *Fix a finite nonempty set L . Fix $(s_\ell)_{\ell \in L}$ satisfying $\sum_{\ell \in L} s_\ell = 1$. Then the corresponding function of $\text{aggregate}_0()$, simply denoted aggregate_0 , satisfies the following.*

- (Continuity.) aggregate_0 is continuous in each coordinate.

- (Increasing.) $\mathbf{aggregate}_0$ is weakly increasing in each c_ℓ , and strictly increasing in c_ℓ if $s_\ell > 0$.
- (Convexity.) For all $((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L})$ in the domain of $\mathbf{aggregate}_0$,

$$\mathbf{aggregate}_0((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) \in \text{conv}\{c_\ell : \ell \in L\}.$$

- (Boundary conditions.) If $s_\ell = 1$, then $\mathbf{aggregate}_0((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L}) = c_\ell$.

Proof. The Continuity, Increasing, and Boundary conditions properties are obvious. For the Convexity property, let $c = \mathbf{aggregate}_0((c_\ell)_{\ell \in L}, (s_\ell)_{\ell \in L})$. Let $c_{\min} = \min\{c_k : k \in L\}$ and $c_{\max} = \max\{c_k : k \in L\}$. Since $\sum_{k \in L} s_k = 1$, it follows that

$$c_{\min} = \prod_{k \in L} c_{\min}^{s_k} \leq \prod_{k \in L} c_k^{s_k} \leq \prod_{k \in L} c_{\max}^{s_k} = c_{\max}.$$

Since $c = \prod_{k \in L} c_k^{s_k}$, it follows that $c \in \text{conv}\{c_k : k \in L\}$. □